SBC-386EX BIOS notes

A few thoughts on the writing of a BIOS for the 386EX 2.1 board. Since the BIOS for the SBC-188 was written, I have written BIOS code for several other boards.

PRELIM

A. Mark Williams Company COHERENT would be a nice small [System V?] Unix clone to get working, but compatibility issues point in the direction of MSDOS or FreeDOS first, running on top of an IBM PC/AT BIOS.

B. Will Sowerbutts says that Linux support for the 386 was abandoned sometime in version 3. However, I'd not be opposed to running an old version of Linux. Just stay off the Internet. :-)

PEARLS

1. Setup options should be retained in the NVRAM. This was learned on the Z180 Mark IV with the UNA BIOS. It was not the case with the SBC-188, which required re-compilation to change a BIOS setting.

2. The keyboard code must be done is close parallel with the IBM PC/AT BIOS. The source code is on the 'net, and I have received a copy of it. I presume IBM published it, as they did with the PC/XT BIOS. Notably, keyboard translate tables MUST be in fixed locations. This may be the problem with MSDOS edit.exe on the VGA3, where a lot of things work, but an annoying few do not.

3. Serial I/O will be the first thing to get working. (It is now in the TEST0 and TEST1 ROM images.) However, the SBC-188 went to great lengths to hook the Video BIOS calls to the serial line, and tried to emulate many Video operations with ANSI (and later, Wyse) escape sequences. I think the way to proceed, is to have a "video" module for each of the display/keyboard combinations supported. The serial emulation "video" is then an interface from the Video BIOS calls to the serial (int 16h) BIOS calls. In practice, the calls would likely be directly to SIO routines.

4. NVRAM setup needs to be less involved than the UNA codes. They are too hard to modify, and too hard to add an additional driver.

5. Debugging on the SBC-188 involved relocating the BIOS to memory, which severely limited memory available to applications. TEST1 already implements a "shadow" mode relocation of the BIOS from ROM to RAM which overlays the ROM at exactly the same F000:0000h locations.

I wish Shadow mode could write-protect the ROM code, but having the ROM in RAM makes setting breakpoints a snap.

6. The SBC-188 debugger supports only 80186/188 instructions. This could be an annoyance in debugging 16-bit ROM code if 386 data registers are use. Using EAX instead of DX:AX is only a data-size prefix away, but I would rather not have to add 386 instructions to the debugger. So, write code for a 186/286 for the most part. And avoid 386 registers as much as possible. (Not forbidden, however.)

7. IBM pulled a sloppy disservice when they used Intel-reserved locations below 00020h for BIOS objects. The 386 has more fault vectors that conflict with hardware & software interrupt trap locations than the earlier processors. This is just something to endure.

8. The 386EX is close to a PC/AT-on-a-chip. Many of the peripherals are likely to be usable in AT-compatible mode. Nice.

9. I'd like to see console support for VGA3, ColorVDU, PropIO, and serial. The lack of interrupts on the PropIO can be simulated off of one of the timers by polling. (This is done on the SBC-188).

TOOLS

A. The SBC-188 BIOS was a mixture of assembly language code and C-code. The Assembler was NASM v. 2.08, now at version 2.08.2. The C-compiler was from Open Watcom 1.9, dated 2010.

B. Code is compiled to the OBJ (16-bit) format, and the ROM image is created from an ordinary EXE file. However, usage of the DATA and BSS sections is strictly off-limits. All DATA/BSS storage MUST take place in the BIOS Data Area at 0x40:0000 .. 0x40:00FF (100h locations). When space is exhausted, then we'll go to an EBDA, location TBD.

C. Be sure you have the Watcom PDF docs. C-code should use the Watcom calling convention: first args passed in AX,BX,CX,DX. SI & DI must be saved & restored if used, AX..DX need not be saved. With the Watcom convention, all external names receive an "_" SUFFIX. Only 'cprintf' uses the '__cdecl' calling convention where all arguments are passed in the stack. It receives an "_" PREFIX, and is declard '__cdecl' in the definition file 'cprintf.h'.

D. Assembly language calling convention: save & restore ALL registers where possible. Good headers on all routines should call out register usage explicitly, especially if some registers get trashed for efficiency reasons (in code that is pounded upon).

E. All source, object, binary, &c. names MUST adhere to the DOS 8.3 naming convention. I would hope that the BIOS could at some later date be maintained on the SBC-386 itself. The SBC-188 code never got this far.

F. Much of the development environment is being pulled from the SBC-188 Makefile. But much is different already.

G. The Bios Data Area [BDA] is defined in NASM assembly as a structure. This means that the following code returns the equipment flag for INT 12h; viz.,

```
struc BDA
     . . .
equip_flag   resw  1          ;equipment flag
     . . .
endstruc
; above is excerpted from the ‘bda.inc’ include file
;
     int12entry:
          push ds
          push 0x40
          pop ds
          mov ax,[equip_flag]
          pop  ds
          iret
```

H. The BDA becomes accessible in a C-program as follows

```
#include “bda.h”
;
; structure declaration in the above, as well as a shortcut
; for accessing vaiables:  dword, word, & byte.
;
     void initialize(void)
     {
          . . .
          bda.equip_flag = system_value;
          . . .
     }
```

I. The directory structire I am working with is:

```
     E:\SBC386
          \ATBIOS        The IBM source code, MASM-like
          \BIOS          working directory
               \outdated      saves daily ZIP archives
```

```
\TEST0          archived TEST0 source & binary
\TEST1          archived TEST1 source & binary
\TOOLS          source, .EXE, Linux.amd64
                HEX2BIN.EXE – new, with CRC's
                BIN2HEX.EXE
                EXE2ROM.EXE – may need updating
                COPT.EXE    – supports rewriting rules
```

The TOOLS are referenced in the BIOS\makefile. Watcom is installed on the C: drive.

2018-03-02 jrc